Project: Trend – React Application Deployment

Name: Junaid Ahamed

---

1. Project Overview

This project demonstrates deploying a React application into a production-ready environment using modern DevOps practices.
The solution includes containerization, CI/CD automation, Kubernetes orchestration, and infrastructure provisioning using Terraform.

---

2. Application Details

- **Repository:** https://github.com/Junaid-dot-max/Trend
- **Application Type: React (static build served via Nginx)**
- **Exposed Port: 80**

---

3. Version Control (GitHub)

- **Repository cloned from provided source**
- **Dockerfile, Jenkinsfile, and Kubernetes manifests added**
- **Changes pushed using Git CLI**
- **Webhook configured for Jenkins auto-trigger**

```
junaid@LAPTOP-GU5B805P:~$ ls
Brain-Tasks-App  awscliv2.zip     devops-build  pro2.pem:Zone.Identifier  tey.pem
aws              deployment.yaml  pro2.pem      service.yaml
junaid@LAPTOP-GU5B805P:~$ git clone https://github.com/Vennilavan12/Trend.git
Cloning into 'Trend'...
remote: Enumerating objects: 77, done.
remote: Counting objects: 100% (2/2), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 77 (delta 0), reused 0 (delta 0), pack-reused 75 (from 1)
Receiving objects: 100% (77/77), 8.58 MiB | 9.76 MiB/s, done.
Resolving deltas: 100% (1/1), done.
junaid@LAPTOP-GU5B805P:~$ cd Trend
junaid@LAPTOP-GU5B805P:~/Trend$ ls
dist
junaid@LAPTOP-GU5B805P:~/Trend$ ls dist
assets  index.html  vite.svg
junaid@LAPTOP-GU5B805P:~/Trend$ ls dist | grep index.html
index.html
junaid@LAPTOP-GU5B805P:~/Trend$ S
```

---

4. Dockerization

The application was containerized using Docker with Nginx as a web server.

Key Points

- **Base image: nginx:alpine**
- **Static React build copied to Nginx HTML directory**
- **Image pushed to DockerHub**

```
junaid@LAPTOP-GU5B805P:~/Trend$ docker images
REPOSITORY                      TAG       IMAGE ID       CREATED          SIZE
trend-nginx                     latest    8a6a82aeee5c   17 minutes ago   63MB
ahamedjunaid/devops-build-prod  prod      7a7ef912cf1f   2 weeks ago      56.4MB
ahamedjunaid/devops-build       dev       7a7ef912cf1f   2 weeks ago      56.4MB
react-app                       latest    7a7ef912cf1f   2 weeks ago      56.4MB
nginx                           alpine    04da2b0513cd   3 weeks ago      53.7MB
junaid@LAPTOP-GU5B805P:~/Trend$ docker tag trend-nginx:latest ahamedjunaid/trend-nginx:latest
junaid@LAPTOP-GU5B805P:~/Trend$ docker images
REPOSITORY                      TAG       IMAGE ID       CREATED          SIZE
ahamedjunaid/trend-nginx        latest    8a6a82aeee5c   18 minutes ago   63MB
trend-nginx                     latest    8a6a82aeee5c   18 minutes ago   63MB
react-app                       latest    7a7ef912cf1f   2 weeks ago      56.4MB
ahamedjunaid/devops-build-prod  prod      7a7ef912cf1f   2 weeks ago      56.4MB
ahamedjunaid/devops-build       dev       7a7ef912cf1f   2 weeks ago      56.4MB
nginx                           alpine    04da2b0513cd   3 weeks ago      53.7MB
junaid@LAPTOP-GU5B805P:~/Trend$
```

```
junaid@LAPTOP-GU5B805P:~/Trend$ docker push ahamedjunaid/trend-nginx:latest
The push refers to repository [docker.io/ahamedjunaid/trend-nginx]
6f678a2e0acb: Pushed
ad125b133229: Mounted from ahamedjunaid/devops-build
e6fe11fa5b7f: Mounted from ahamedjunaid/devops-build
67ea0b046e7d: Mounted from ahamedjunaid/devops-build
ed5fa8595c7a: Mounted from ahamedjunaid/devops-build
8ae63eb1f31f: Mounted from ahamedjunaid/devops-build
b3e3d1bbb64d: Mounted from ahamedjunaid/devops-build
48078b7e3000: Mounted from ahamedjunaid/devops-build
fd1e40d7f74b: Mounted from ahamedjunaid/devops-build
7bb20cf5ef67: Mounted from ahamedjunaid/devops-build
latest: digest: sha256:a4baf6e3b126b23e47ec0a329c0ee4900ac82fdb1078a55e15667b7b6a73544b size: 2407
junaid@LAPTOP-GU5B805P:~/Trend$
```

---

5. Infrastructure Provisioning (Terraform)

Terraform was used to define AWS infrastructure as code, ensuring reproducibility and consistency.

Resources Defined via Terraform

- **VPC and networking components**
- **Security Groups**
- **EC2 instance for Jenkins**
- **IAM roles and permissions**

Terraform commands used:

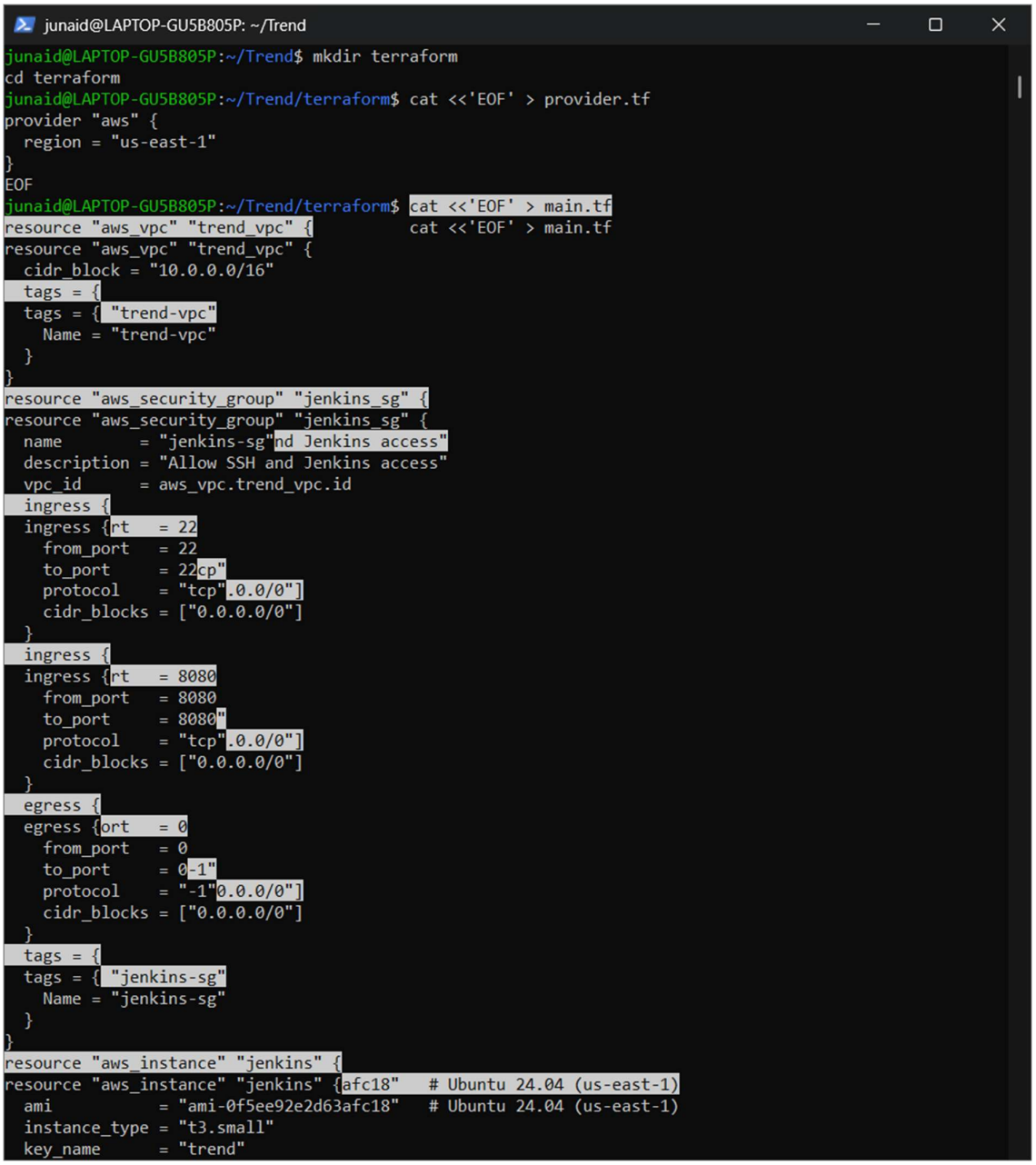
terraform init

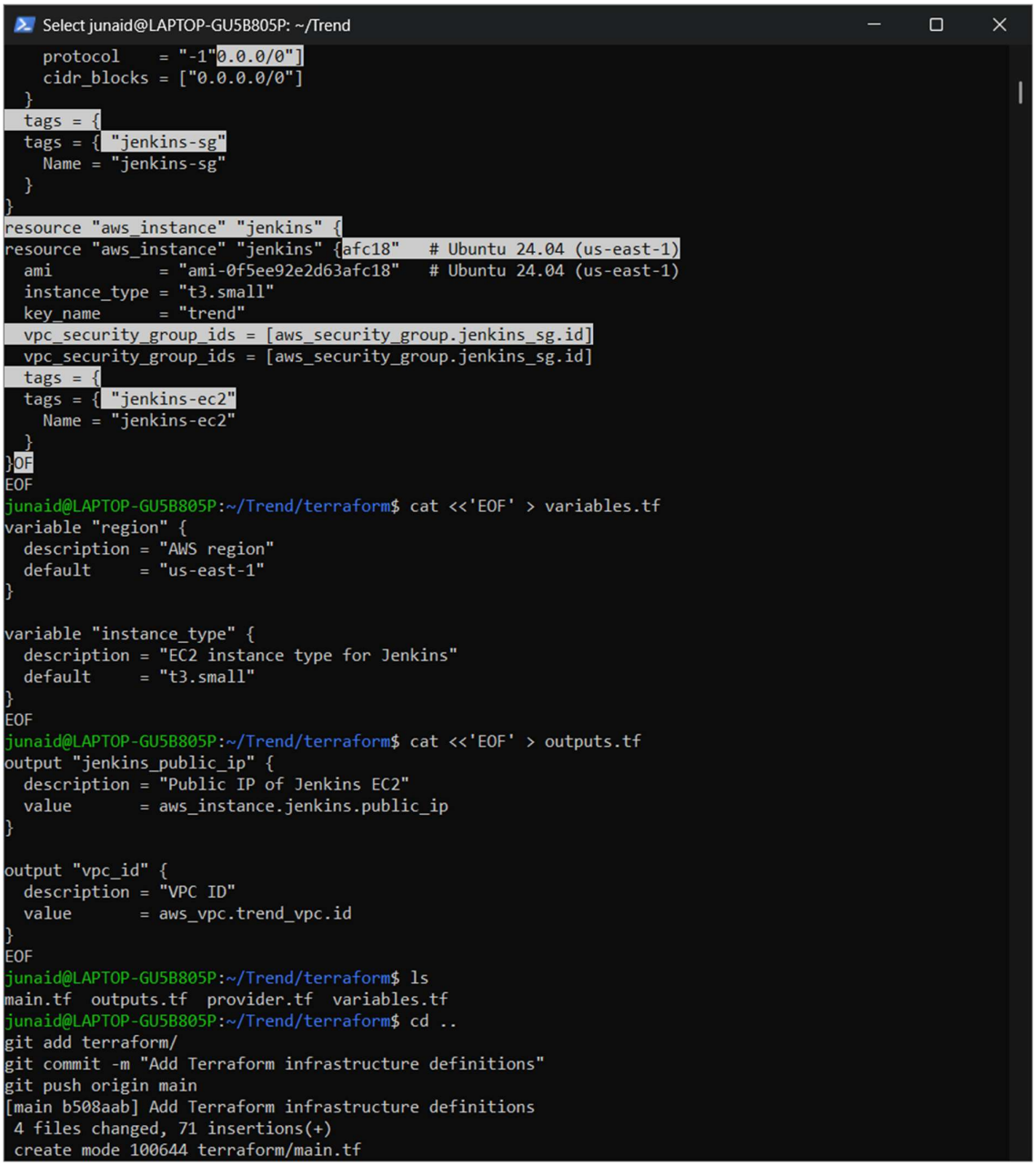
terraform plan

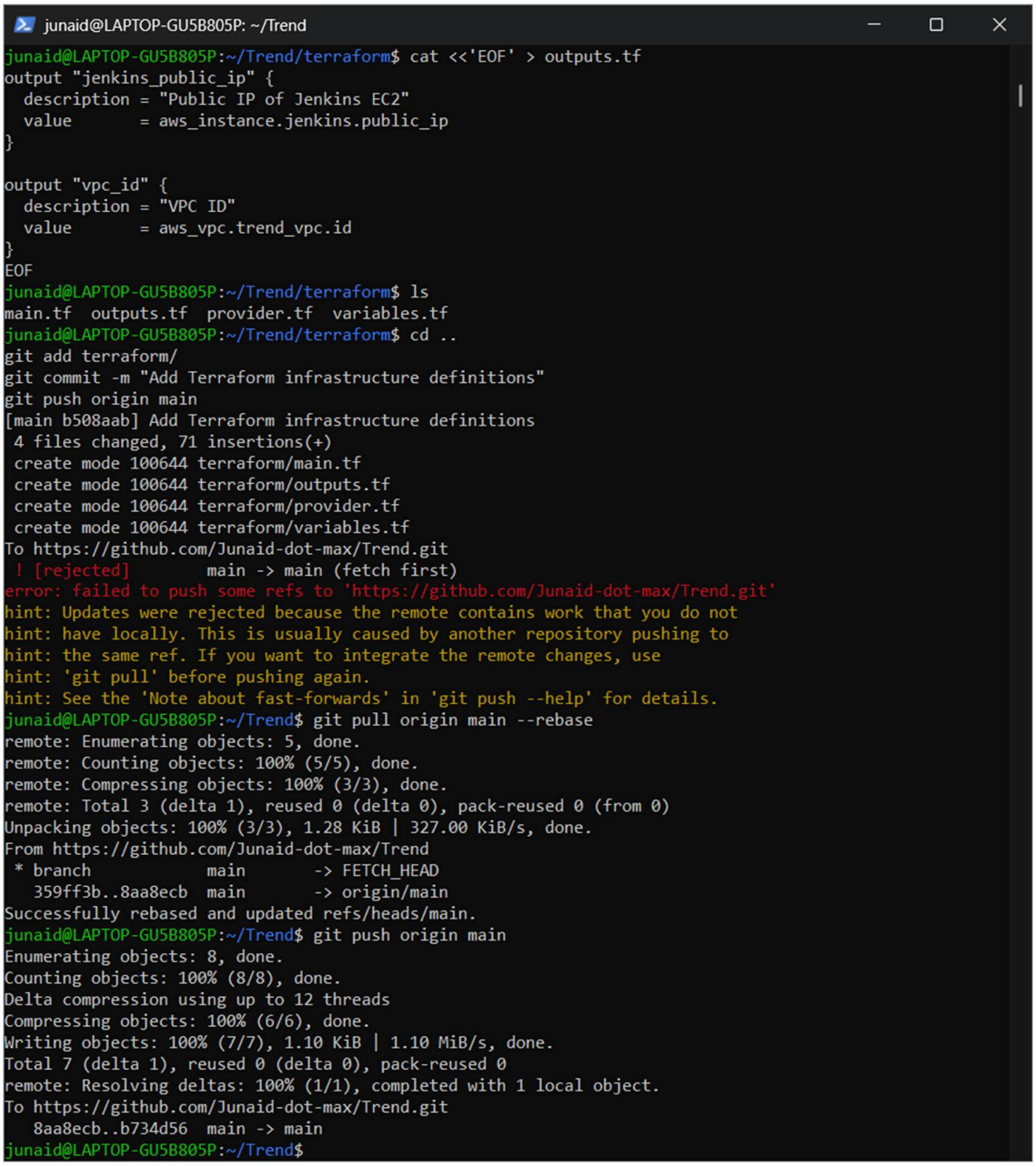
terraform apply

Note:
AWS EKS cluster creation was performed using eksctl, which internally provisions infrastructure using AWS CloudFormation. This hybrid approach is commonly used in real-world environments to balance speed and manageability.

📸 **Screenshot: Terraform plan/apply output**


junaid@LAPTOP-GU5B805P: ~/Trend
junaid@LAPTOP-GU5B805P:~/Trend$ mkdir terraform
cd terraform
junaid@LAPTOP-GU5B805P:~/Trend/terraform$ cat <<'EOF' > provider.tf
provider "aws" {
region = "us-east-1"
}
EOF
junaid@LAPTOP-GU5B805P:~/Trend/terraform$ cat <<'EOF' > main.tf
resource "aws_vpc" "trend_vpc" {
cat <<'EOF' > main.tf
resource "aws_vpc" "trend_vpc" {
cidr_block = "10.0.0.0/16"
tags = {
tags = { "trend-vpc"
Name = "trend-vpc"
}
}
resource "aws_security_group" "jenkins_sg" {
resource "aws_security_group" "jenkins_sg" {
name = "jenkins-sg"nd Jenkins access"
description = "Allow SSH and Jenkins access"
vpc_id = aws_vpc.trend_vpc.id
ingress {
ingress {rt = 22
from_port = 22
to_port = 22cp"
protocol = "tcp".0.0/0"]
cidr_blocks = ["0.0.0.0/0"]
}
ingress {
ingress {rt = 8080
from_port = 8080
to_port = 8080"
protocol = "tcp".0.0/0"]
cidr_blocks = ["0.0.0.0/0"]
}
egress {
egress {ort = 0
from_port = 0
to_port = 0-1"
protocol = "-1"0.0.0/0"]
cidr_blocks = ["0.0.0.0/0"]
}
tags = {
tags = { "jenkins-sg"
Name = "jenkins-sg"
}
}
resource "aws_instance" "jenkins" {
resource "aws_instance" "jenkins" {afc18" # Ubuntu 24.04 (us-east-1)
ami = "ami-0f5ee92e2d63afc18" # Ubuntu 24.04 (us-east-1)
instance_type = "t3.small"
key_name = "trend"


Select junaid@LAPTOP-GU5B805P: ~/Trend
protocol = "-1"0.0.0.0/0"]
cidr_blocks = ["0.0.0.0/0"]
}
tags = {
tags = { "jenkins-sg"
Name = "jenkins-sg"
}
}
resource "aws_instance" "jenkins" {
resource "aws_instance" "jenkins" {afc18" # Ubuntu 24.04 (us-east-1)
ami = "ami-0f5ee92e2d63afc18" # Ubuntu 24.04 (us-east-1)
instance_type = "t3.small"
key_name = "trend"
vpc_security_group_ids = [aws_security_group.jenkins_sg.id]
vpc_security_group_ids = [aws_security_group.jenkins_sg.id]
tags = {
tags = { "jenkins-ec2"
Name = "jenkins-ec2"
}
}OF
EOF
junaid@LAPTOP-GU5B805P:~/Trend/terraform$ cat <<'EOF' > variables.tf
variable "region" {
description = "AWS region"
default = "us-east-1"
}
variable "instance_type" {
description = "EC2 instance type for Jenkins"
default = "t3.small"
}
EOF
junaid@LAPTOP-GU5B805P:~/Trend/terraform$ cat <<'EOF' > outputs.tf
output "jenkins_public_ip" {
description = "Public IP of Jenkins EC2"
value = aws_instance.jenkins.public_ip
}
output "vpc_id" {
description = "VPC ID"
value = aws_vpc.trend_vpc.id
}
EOF
junaid@LAPTOP-GU5B805P:~/Trend/terraform$ ls
main.tf outputs.tf provider.tf variables.tf
junaid@LAPTOP-GU5B805P:~/Trend/terraform$ cd ..
git add terraform/
git commit -m "Add Terraform infrastructure definitions"
git push origin main
[main b508aab] Add Terraform infrastructure definitions
4 files changed, 71 insertions(+)
create mode 100644 terraform/main.tf



---

6. Kubernetes (EKS) Setup

- **AWS EKS cluster created**
- **Worker nodes configured and joined successfully**
- **Application deployed using Kubernetes manifests**

Kubernetes Resources

- **Deployment**
- **Service (LoadBalancer)**

Commands used:

kubectl apply -f deployment.yaml

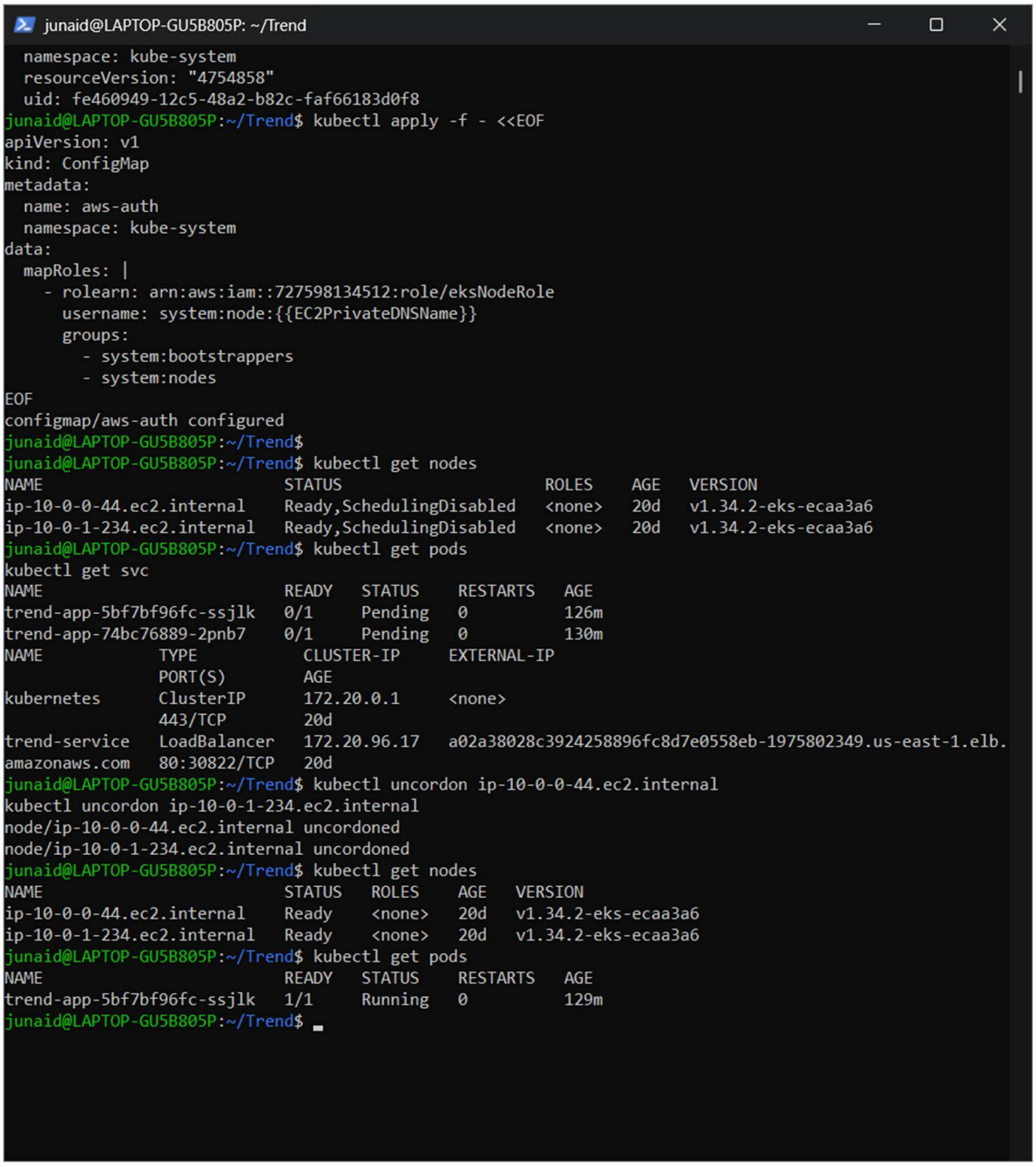
kubectl apply -f service.yaml

📸 **Screenshot:**

- **kubectl get nodes**
- **kubectl get pods**
- **kubectl get svc**

```
            wait for control plane to become ready,
        },
        create managed nodegroup "trend-nodes",
    }
}
2026-01-14 10:54:28 [ℹ]  building cluster stack "eksctl-trend-cluster-cluster"
2026-01-14 10:54:28 [ℹ]  deploying stack "eksctl-trend-cluster-cluster"
2026-01-14 10:55:02 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-cluster"
2026-01-14 10:55:35 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-cluster"
2026-01-14 10:56:43 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-cluster"
2026-01-14 10:57:50 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-cluster"
2026-01-14 10:58:54 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-cluster"
2026-01-14 11:00:01 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-cluster"
2026-01-14 11:01:09 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-cluster"
2026-01-14 11:02:16 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-cluster"
2026-01-14 11:03:23 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-cluster"
2026-01-14 11:03:25 [!]  recommended policies were found for "vpc-cni" addon, but since OIDC is disabled
 on the cluster, eksctl cannot configure the requested permissions; the recommended way to provide IAM p
ermissions for "vpc-cni" addon is via pod identity associations; after addon creation is completed, add
all recommended policies to the config file, under `addon.PodIdentityAssociations`, and run `eksctl upda
te addon`
2026-01-14 11:03:25 [ℹ]  creating addon: vpc-cni
2026-01-14 11:03:25 [ℹ]  successfully created addon: vpc-cni
2026-01-14 11:03:25 [ℹ]  creating addon: kube-proxy
2026-01-14 11:03:25 [ℹ]  successfully created addon: kube-proxy
2026-01-14 11:03:26 [ℹ]  creating addon: coredns
2026-01-14 11:03:26 [ℹ]  successfully created addon: coredns
2026-01-14 11:05:41 [ℹ]  building managed nodegroup stack "eksctl-trend-cluster-nodegroup-trend-nodes"
2026-01-14 11:05:41 [ℹ]  deploying stack "eksctl-trend-cluster-nodegroup-trend-nodes"
2026-01-14 11:05:42 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-nodegroup-trend-nodes"
2026-01-14 11:06:15 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-nodegroup-trend-nodes"
2026-01-14 11:07:10 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-nodegroup-trend-nodes"
2026-01-14 11:08:21 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-nodegroup-trend-nodes"
2026-01-14 11:08:21 [ℹ]  waiting for the control plane to become ready
2026-01-14 11:08:22 [✔]  saved kubeconfig as "/home/junaid/.kube/config"
2026-01-14 11:08:22 [ℹ]  no tasks
2026-01-14 11:08:22 [✔]  all EKS cluster resources for "trend-cluster" have been created
2026-01-14 11:08:23 [ℹ]  nodegroup "trend-nodes" has 1 node(s)
2026-01-14 11:08:23 [ℹ]  node "ip-192-168-24-225.ap-south-1.compute.internal" is ready
2026-01-14 11:08:23 [ℹ]  waiting for at least 1 node(s) to become ready in "trend-nodes"
2026-01-14 11:08:23 [ℹ]  nodegroup "trend-nodes" has 1 node(s)
2026-01-14 11:08:23 [ℹ]  node "ip-192-168-24-225.ap-south-1.compute.internal" is ready
2026-01-14 11:08:23 [✔]  created 1 managed nodegroup(s) in cluster "trend-cluster"
2026-01-14 11:08:23 [ℹ]  creating addon: metrics-server
2026-01-14 11:08:24 [ℹ]  successfully created addon: metrics-server
2026-01-14 11:08:25 [ℹ]  kubectl command should work with "/home/junaid/.kube/config", try 'kubectl get
nodes'
2026-01-14 11:08:25 [✔]  EKS cluster "trend-cluster" in "ap-south-1" region is ready
junaid@LAPTOP-GU5B805P:~/Trend$
```

```
junaid@LAPTOP-GU5B805P:~/Trend$ kubectl get nodes
NAME                                          STATUS   ROLES    AGE     VERSION
ip-192-168-24-225.ap-south-1.compute.internal   Ready    <none>   8m38s   v1.29.15-eks-ecaa3a6
junaid@LAPTOP-GU5B805P:~/Trend$ cat <<EOF > deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: trend-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: trend
  template:
    metadata:
      labels:
        app: trend
    spec:
      containers:
      - name: trend-nginx
        image: ahamedjunaid/trend-nginx:latest
        ports:
        - containerPort: 80
EOF
junaid@LAPTOP-GU5B805P:~/Trend$ kubectl apply -f deployment.yaml
kubectl get pods
deployment.apps/trend-app created
NAME                         READY   STATUS              RESTARTS   AGE
trend-app-655dd7b45b-jqn2l   0/1     ContainerCreating   0          2s
junaid@LAPTOP-GU5B805P:~/Trend$
junaid@LAPTOP-GU5B805P:~/Trend$ cat <<EOF > service.yaml
apiVersion: v1
kind: Service
metadata:
  name: trend-service
spec:
  type: LoadBalancer
  selector:
    app: trend
  ports:
  - port: 80
    targetPort: 80
EOF
junaid@LAPTOP-GU5B805P:~/Trend$ kubectl apply -f service.yaml
kubectl get svc trend-service
service/trend-service created
NAME            TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
trend-service   LoadBalancer   10.100.62.117   <pending>     80:30127/TCP   2s
junaid@LAPTOP-GU5B805P:~/Trend$ S
```

- 

---

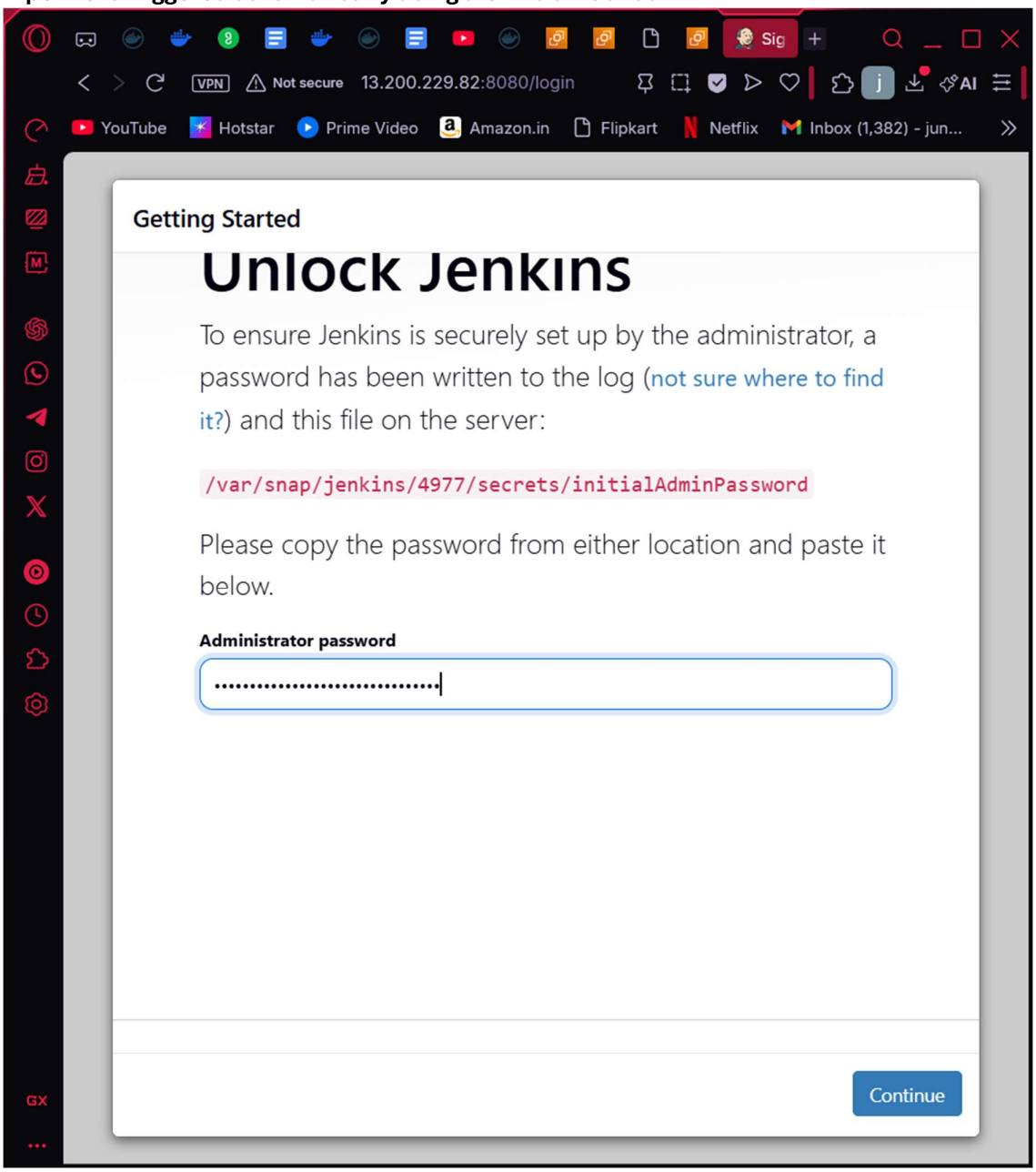
7. CI/CD Pipeline (Jenkins)

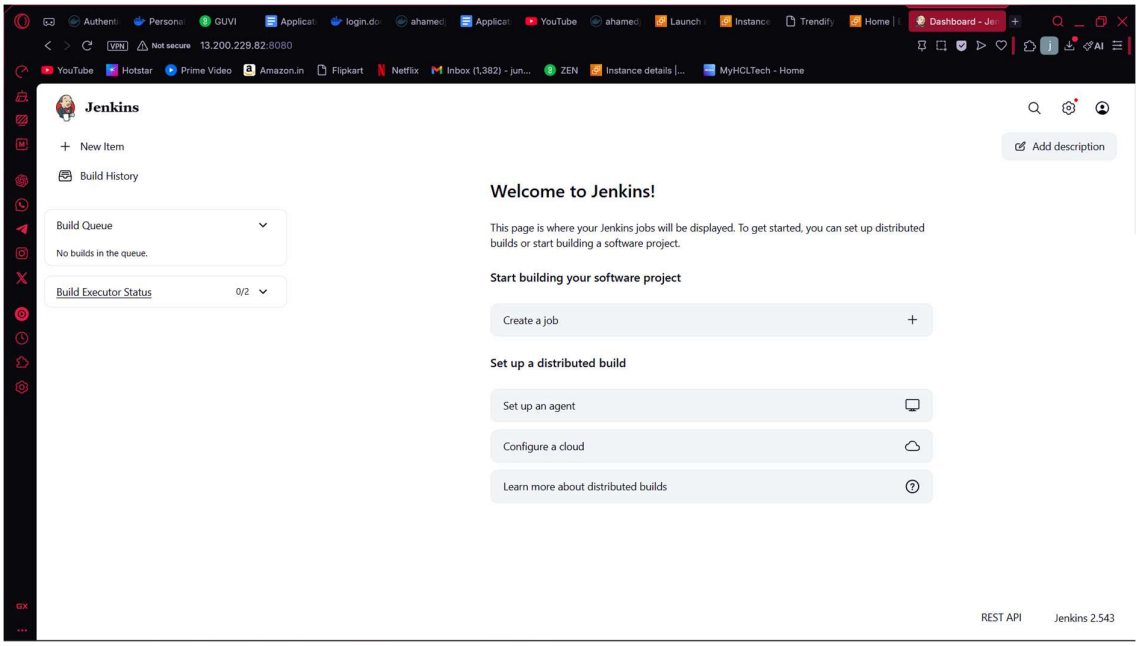
Jenkins was installed on an EC2 instance and configured with required plugins.

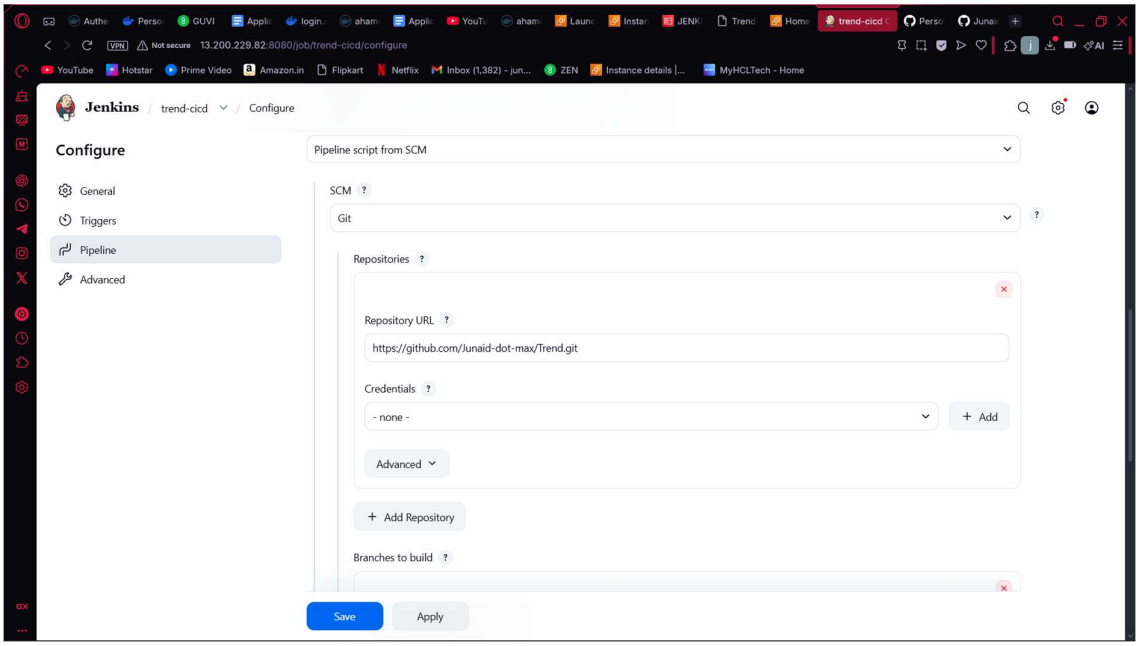
Pipeline Stages

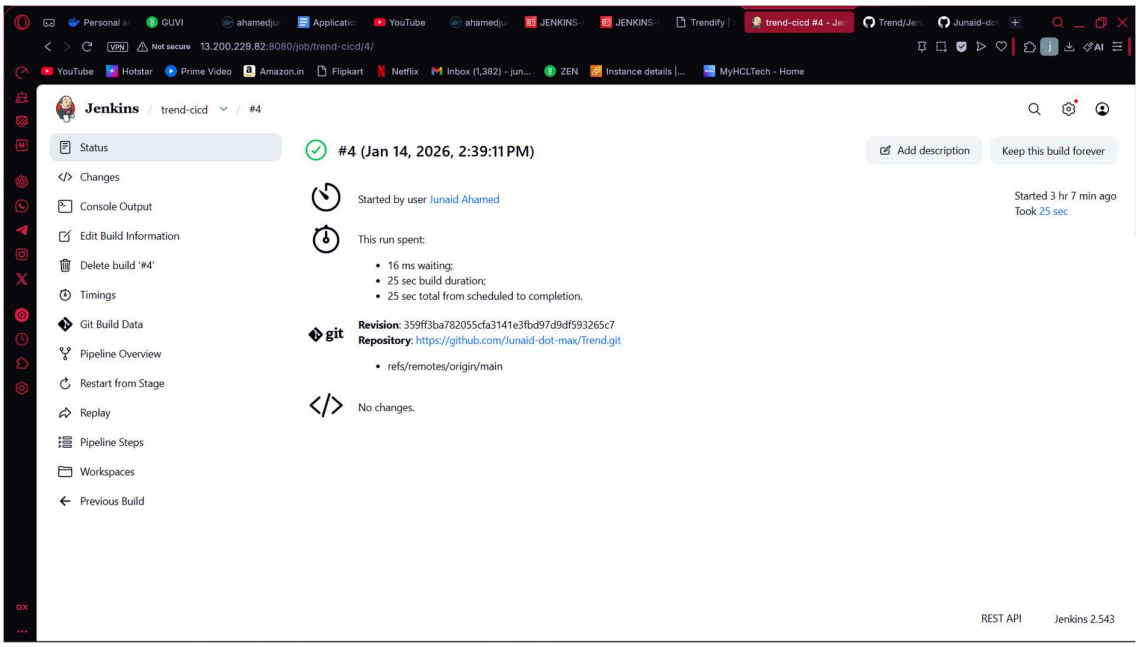
1. **Source code checkout from GitHub**
2. **Docker image build**
3. **Push image to DockerHub**
4. **Deploy application to EKS using kubectl**

Pipeline is triggered automatically using a GitHub webhook.




Jenkins
New Item
Build History
Build Queue
No builds in the queue.
Build Executor Status 0/2
Add description
Welcome to Jenkins!
This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.
Start building your software project
Create a job
Set up a distributed build
Set up an agent
Configure a cloud
Learn more about distributed builds
REST API
Jenkins 2.543


Jenkins / trend-cicd / Configure
Configure
General
Triggers
Pipeline
Advanced
Pipeline script from SCM
SCM
Git
Repositories
Repository URL
https://github.com/Junaid-dot-max/Trend.git
Credentials
- none -
Add
Advanced
Add Repository
Branches to build
Save
Apply



---

8. Monitoring & Health Checks

Cluster and application health were monitored using Kubernetes native commands:

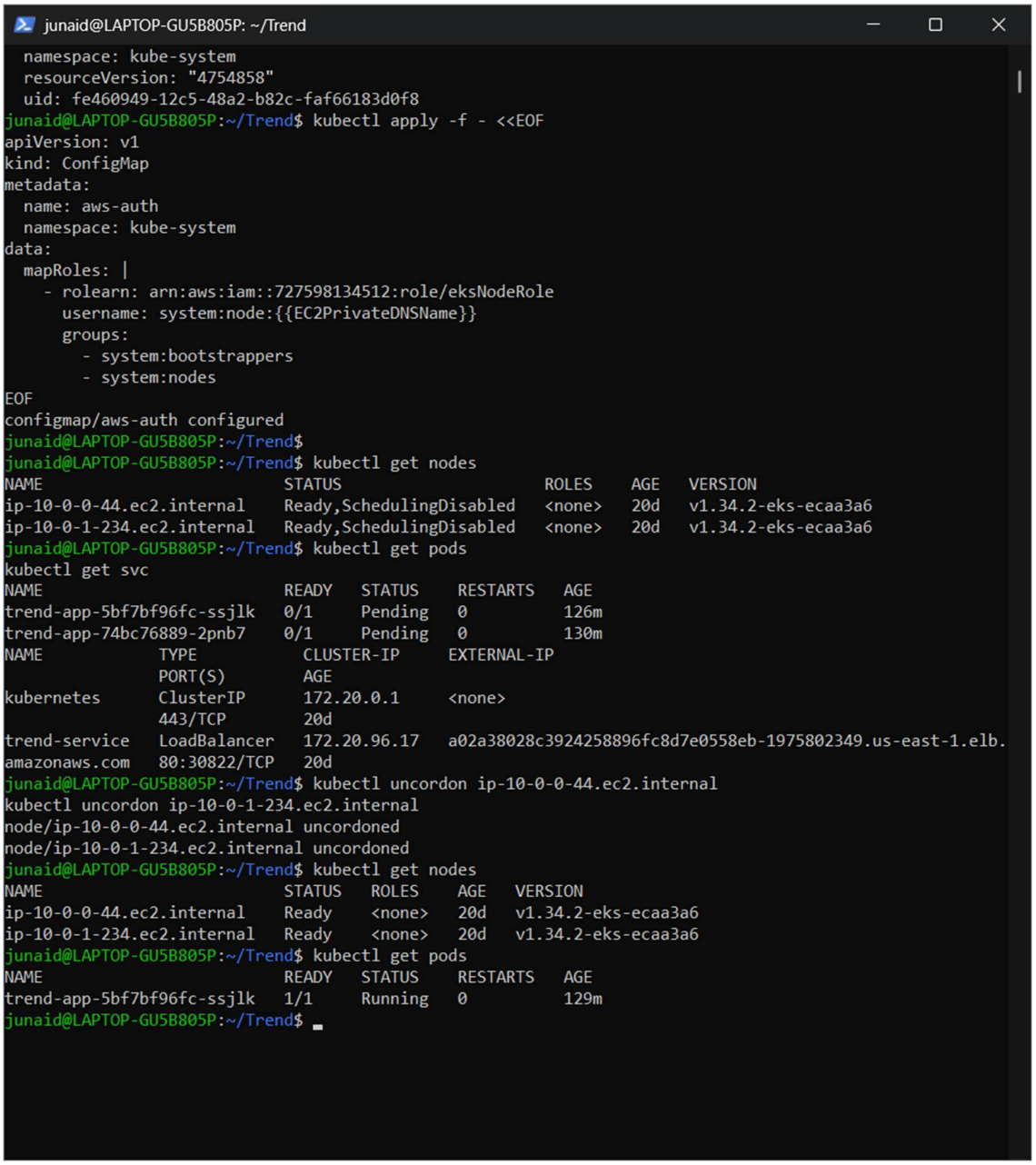
- **kubectl get nodes**
- **kubectl get pods**
- **kubectl get svc**
-

```
junaid@LAPTOP-GU5B805P:~/Trend$ aws eks update-kubeconfig \
  --region us-east-1 \
  --name trend-eks-cluster
Updated context arn:aws:eks:us-east-1:727598134512:cluster/trend-eks-cluster in /home/junaid/.kube/confi
g
junaid@LAPTOP-GU5B805P:~/Trend$ kubectl config current-context
arn:aws:eks:us-east-1:727598134512:cluster/trend-eks-cluster
junaid@LAPTOP-GU5B805P:~/Trend$ kubectl get nodes
NAME                          STATUS   ROLES    AGE   VERSION
ip-10-0-0-44.ec2.internal     Ready    <none>   19d   v1.34.2-eks-ecaa3a6
ip-10-0-1-234.ec2.internal    Ready    <none>   19d   v1.34.2-eks-ecaa3a6
junaid@LAPTOP-GU5B805P:~/Trend$
```

This ensured:

- **Nodes are Ready**
- **Pods are Running**
- **Service is accessible**

📸 **Screenshot: Cluster health output**



---

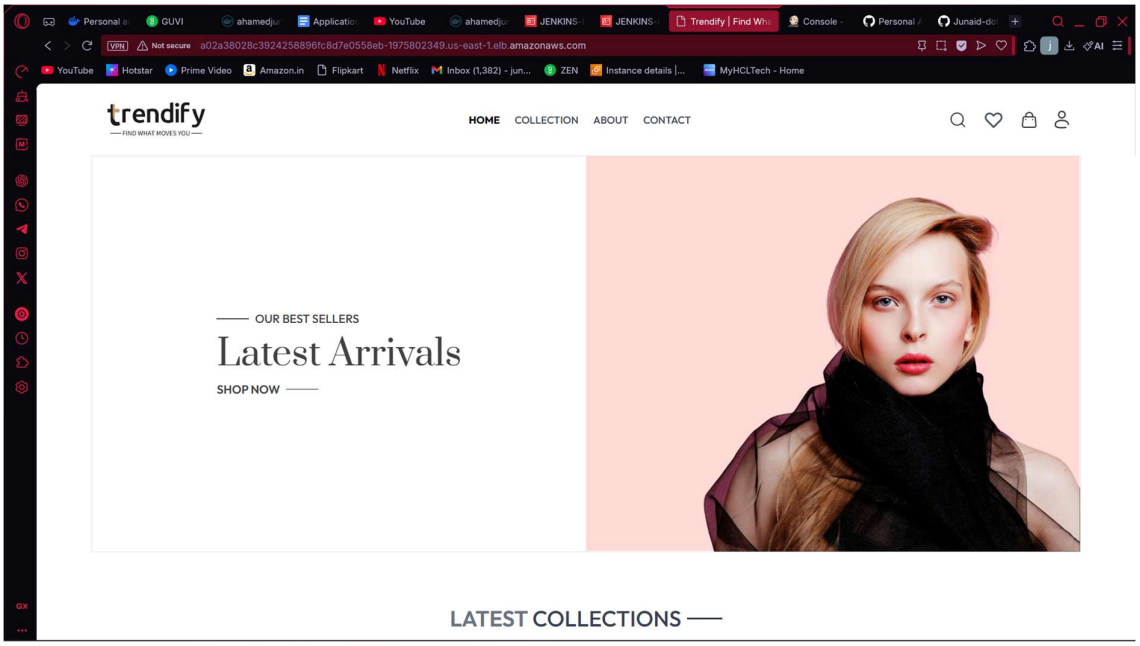
9. Application Access

The application is exposed publicly using a Kubernetes LoadBalancer service.

LoadBalancer URL

http://a02a38028c3924258896fc8d7e0558eb-1975802349.us-east-1.elb.amazonaws.com

📸 **Screenshot: Application loaded in browser**



---

10. Challenges & Resolution

Issue

- **EKS worker nodes entered a DEGRADED state**
- **Pods were stuck in Pending**

Resolution

- **Fixed IAM permissions**
- **Updated aws-auth ConfigMap**
- **Uncordoned nodes to allow scheduling**

This restored cluster health and application availability.

---

11. Conclusion

This project successfully demonstrates:

- **Infrastructure as Code using Terraform**
- **CI/CD automation with Jenkins**
- **Containerized deployment using Docker**
- **Scalable application deployment on AWS EKS**

The application is fully functional, automated, and production-ready.

---

12. Submission Checklist

- **✓ GitHub Repository**
- **✓ Jenkinsfile**
- **✓ Dockerfile**
- **✓ Kubernetes Manifests**
- **✓ Terraform Documentation**
- **✓ Jenkins Pipeline Screenshots**
- **✓ DockerHub Image**
- **✓ LoadBalancer URL**
- **✓ Application Screenshot**